

Checkpoint 5 — k-means Objective & PCA

INFO 521 — Machine Learning Foundations

i Course-schedule placement

Weeks 6–7 · with Modules 6–7 — Clustering & PCA · gates Project 2.3 / Gate B.

Gates: Project 2.3 (Clustering) — and with it, Gate B. **Format:** in-class, closed-book, ~15–20 min, one derivation/explanation. Graded Satisfactory / Not-Yet. **Retakeable** (an alternate version is provided below).

This checkpoint carries the unsupervised-gate recall outcomes: the k-means objective and why model selection for K is non-trivial (Version A), and the PCA procedure and the meaning of its eigenstructure (Version B).

Prompt (Version A)

State the **k-means objective** J as a function of the cluster assignments and centroids. Explain why choosing K by **minimizing J over K** trivially fails, and describe **one principled method** for selecting K .

Prompt (Version B — retake)

Describe the **PCA procedure** step by step for a centered data matrix, and state what the **eigenvalues** and **eigenvectors** of the covariance matrix represent.

Satisfactory criteria

- (A) Correct objective J (within-cluster sum of squared distances); states that J is **non-increasing in K** and hits 0 at $K = N$, so minimizing J always prefers more clusters; names a valid criterion (elbow, silhouette, or gap) and describes it.
- (B) Lists center $\rightarrow$ covariance $\rightarrow$ eigendecompose (or SVD) $\rightarrow$ sort $\rightarrow$ project; states eigenvectors = principal directions (orthogonal axes of maximal variance), eigenvalues = variance along each, and the ratio $\lambda_i / \sum_j \lambda_j$ = proportion of variance explained.

Answer key

A: With assignments $z_n \in \{1, \dots, K\}$ and centroids μ_k ,

$$J = \sum_{n=1}^N \|\mathbf{x}_n - \mu_{z_n}\|^2 = \sum_{k=1}^K \sum_{n \in C_k} \|\mathbf{x}_n - \mu_k\|^2.$$

J can only **decrease** as K grows (more centroids $\Rightarrow$ every point can sit nearer one), reaching $J = 0$ when $K = N$ (each point is its own cluster). So “minimize J over K ” degenerately selects $K =$

N and is useless as a selection rule. Pick K instead by, e.g., the **elbow** (the K past which J falls only marginally) or the **silhouette** (choose the K maximizing the mean silhouette $s_n = (b_n - a_n) / \max(a_n, b_n)$, where a_n/b_n are the mean intra-cluster and nearest-other-cluster distances).

B: (1) **Center** the data, $\mathbf{X}_c = \mathbf{X} - \bar{\mathbf{x}}$ (standardize if features differ in scale). (2) Form the **covariance** $\Sigma = \frac{1}{N-1} \mathbf{X}_c^\top \mathbf{X}_c$. (3) **Eigendecompose** $\Sigma = \mathbf{V} \Lambda \mathbf{V}^\top$ (or take the SVD $\mathbf{X}_c = \mathbf{U} \mathbf{S} \mathbf{V}^\top$, with $\lambda_i = s_i^2 / (N - 1)$). (4) **Sort** components by descending eigenvalue. (5) **Project** onto the top k : $\mathbf{Z} = \mathbf{X}_c \mathbf{V}_k$. The **eigenvectors** are the principal directions — orthogonal axes of greatest variance; each **eigenvalue** λ_i is the variance captured along its direction, and $\lambda_i / \sum_j \lambda_j$ is the proportion of total variance that component explains.